

TourPlanner — Final Protocol

Source markdown for the final protocol PDF. Render with VSCode's Markdown preview (which understands Mermaid out-of-the-box with the Markdown Preview Mermaid Support extension), with `npx md-to-pdf / pandoc` , or by pasting into a Markdown-aware editor and exporting as PDF.

1. Application Overview

TourPlanner is a web-based tour management application for bike, hike, running, and walking tours. After registering and logging in, the user creates tours that are routed through OpenRouteService, records tour logs as the tour is accomplished, and sees the computed popularity and child-friendliness of each tour. Tours and logs are searchable by full text — including over the computed values — and the whole catalogue can be exported and re-imported as JSON. Live weather at each tour's destination is fetched from OpenWeatherMap.

Technology stack

Layer	Technology
Frontend	Angular 19 (standalone components, signals, zoneless CD)
Backend	Spring Boot 3.4 (Web, Data JPA, Security, Validation)
Auth	JWT (jjwt 0.12) + BCrypt password hashing
Database	PostgreSQL 16 via JPA / Hibernate
Routing	OpenRouteService directions + geocoding API
Map	Leaflet 1.9 with OpenStreetMap tiles
Weather	OpenWeatherMap API (graceful fallback to stub data)
Logging	SLF4J (Lombok @Slf4j) over Spring Boot's Logback backend
Config	Externalized in application.yml via environment variables
Tests	JUnit 5, Mockito, Spring MockMvc — 36 backend tests

2. Application Architecture

2.1 Layer architecture

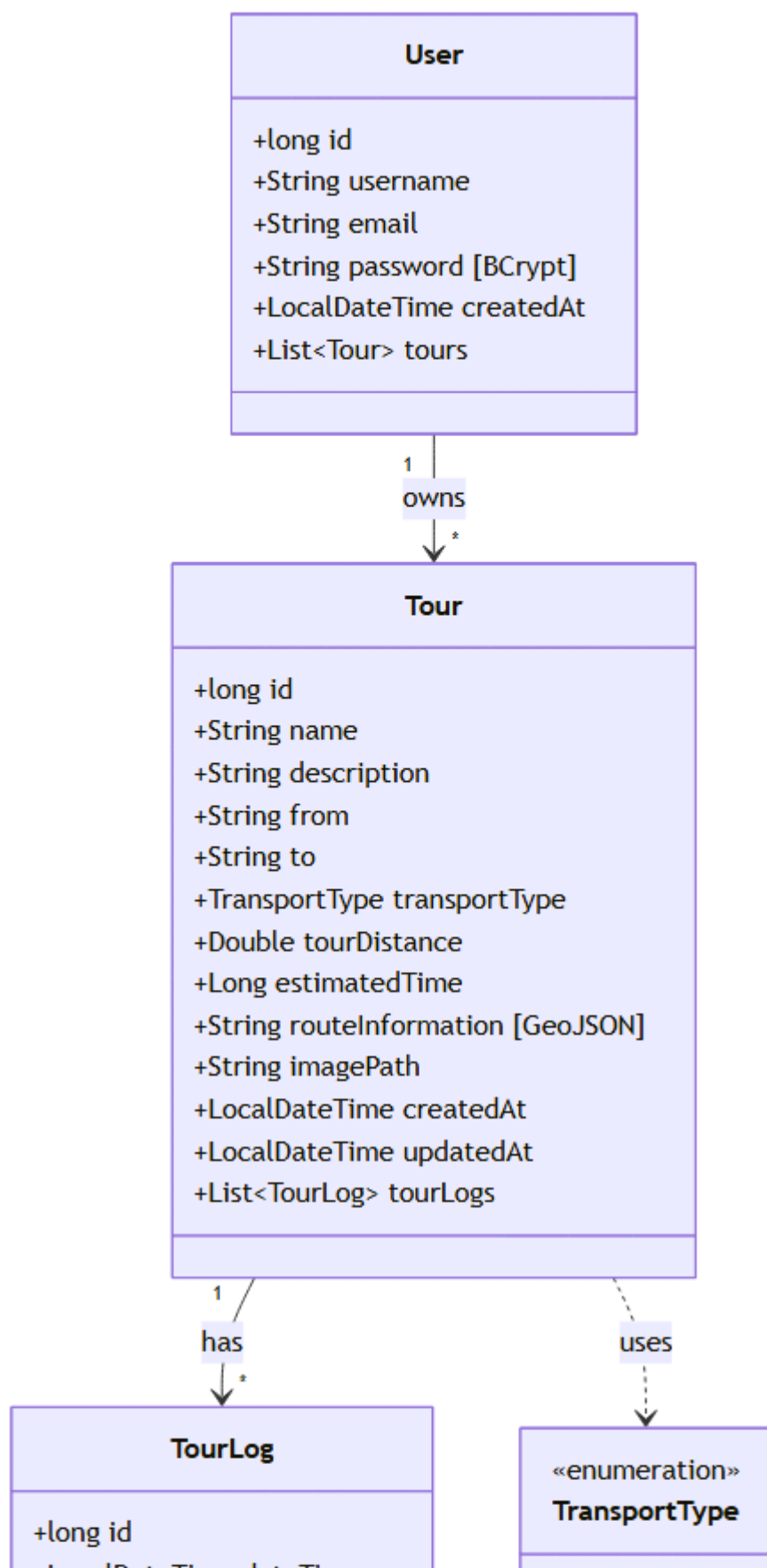
The backend follows a strict three-layer architecture; each layer only knows the layer directly below it. Cross-cutting concerns (security, exception handling) sit beside the layered flow.

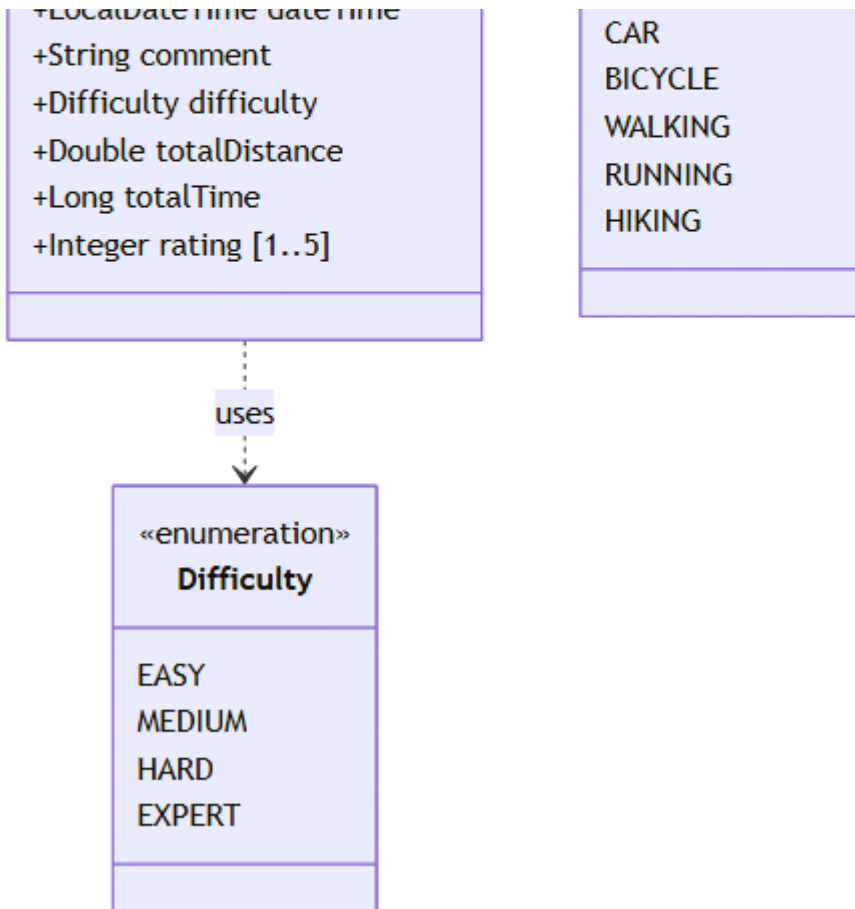


The controller layer never reads from `JpaRepository` directly — it depends only on the service layer (`TourService` , `AuthService` , `TourLogService`). The service layer is the only place that talks to repositories and

to external services (`OpenRouteService` , `WeatherService`). The DTO mapper at the controller / service boundary keeps entities out of the HTTP surface area.

2.2 Domain class diagram





2.3 Backend package map

```

org.example.tourplanner
├── controller/                (REST API + GlobalExceptionHandler)
│   ├── AuthController        POST /api/auth/{login,register}
│   ├── TourController        /api/tours, /api/tours/{id}, /image, /export, /import
│   ├── TourLogController     /api/tours/{tourId}/logs/...
│   └── GlobalExceptionHandler ResourceNotFound -> 404, BadRequest -> 400, validation -> 400
├── service/                  (business logic + external integrations)
│   ├── TourService           CRUD, full-text search, import/export, image storage
│   ├── TourLogService
│   ├── AuthService           register, login, JWT issuance
│   ├── OpenRouteService      directions + geocoding (Strategy by transport type)
│   └── WeatherService         OpenWeatherMap (with stub fallback) ← unique feature
├── repository/              (Spring Data JPA)
│   ├── TourRepository        findById + JPQL searchByUserAndQuery
│   ├── TourLogRepository
│   └── UserRepository
├── model/                   (JPA entities)
│   ├── Tour                  @Entity tours
│   ├── TourLog               @Entity tour_logs
│   └── User                   @Entity users
  
```

```

├─ dto/                                (request/response shapes + mapper)
│   ├── TourRequest / TourResponse
│   ├── TourLogRequest / TourLogResponse
│   ├── LoginRequest / RegisterRequest / AuthResponse
│   ├── WeatherResponse
│   ├── TourExportDto                (import/export bundle)
│   └─ DtoMapper                    entity ↔ DTO + computed attributes
├─ security/                          (cross-cutting)
│   ├── SecurityConfig              stateless filter chain, JWT auth, BCrypt
│   ├── JwtTokenProvider            sign / verify HS512 tokens
│   ├── JwtAuthenticationFilter     once-per-request token extraction
│   └─ UserDetailsServiceImpl       loads user by email

```

2.4 Frontend (MVVM) layer

Each feature has a dedicated ViewModel that owns all mutable state as Angular signals. Components are thin views; templates bind one-way to ViewModel signals.

```

frontend/src/app
├─ core/
│   ├── models.ts                  interfaces (Tour, TourLog, User, WeatherData, ...)
│   ├── auth.utils.ts              functional authGuard + authInterceptor
│   └─ services/
│       ├── auth.service.ts        register, login, token storage, isAuthenticated
│       └─ tour.service.ts         tour CRUD, logs CRUD, weather, import, export
├─ features/
│   ├── auth/
│   │   ├── login.component.ts     (View)
│   │   └─ login.viewmodel.ts     (signals: isLogin, name, email, ..., error)
│   └─ tours/
│       ├── dashboard/
│       │   ├── dashboard.component.ts
│       │   └─ dashboard.viewmodel.ts (signals: tours, searchQuery, error; onExport,
onImport)
│       ├── tour-detail/
│       │   ├── tour-detail.component.ts
│       │   └─ tour-detail.viewmodel.ts (signals: tour, logs, weather; log form state)
│       └─ tour-form/
│           ├── tour-form.component.ts
│           └─ tour-form.viewmodel.ts (signals: form fields, image preview, loading)
└─ shared/
    ├── navbar.component.ts        (used on dashboard, tour-detail, tour-form)
    ├── tour-map.component.ts      (Leaflet wrapper, @Input routeGeoJson, fromLocation,
toLocation, interactive)
    ├── auth-image.pipe.ts         (adds Bearer token to image URLs)
    └─ format.utils.ts             (formatDistance, formatTime, getTransportLabel)

```

2.5 Cross-cutting concerns: logging & configuration

Logging. The backend logs through SLF4J, bound to Spring Boot's default Logback backend. Lombok's `@Slf4j` injects a `log` field into the three classes that perform external I/O — `OpenRouteService`, `WeatherService`, and `TourService` — so that every failure mode at an integration boundary leaves a trace without leaking into the happy path:

- `OpenRouteService` — missing API key (`log.error`), failed directions call, failed geocoding.
- `WeatherService` — missing API key (`log.warn` , then stub fallback), failed weather call.
- `TourService` — geocoding failure during weather lookup (`log.warn`), skipped duplicate on import (`log.info`), ORS failure during import (`log.warn`).

Log levels are configured in `application.yml` (`org.example.tourplanner: INFO`), so verbosity is tuned by configuration rather than code. The deliberate choice was to log at the boundary where an external system can fail and to pick the level by severity — `error` when a required integration is unusable, `warn` when the app degrades gracefully, `info` for noteworthy business events like skipped imports.

Configuration. No connection string, secret, or base directory is hard-coded. Every environment-specific value in `application.yml` is an `${ENV_VAR:default}` placeholder — `DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USER` / `DB_PASSWORD` for the datasource, `JWT_SECRET`, `ORS_API_KEY`, `WEATHER_API_KEY`, and `UPLOAD_DIR` for the externally-stored tour images. The committed defaults are development-only fallbacks; a real deployment overrides them through the environment, keeping production credentials out of the source tree.

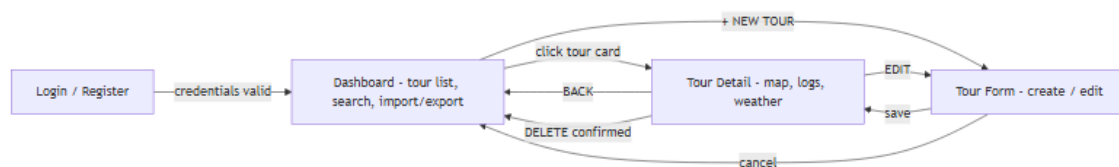
3. Use Cases



`Register` and `Login` are unauthenticated; everything else requires a valid JWT in the `Authorization: Bearer ...` header. Tours and tour logs are scoped to the owning user; no sharing. The CRUD operations on tours and tour logs each include client-side and server-side validation, so wrong input never crashes the application.

4. UI Flow & Wireframes

The application has four screens. Login is the entry point; the dashboard is the central hub; tour detail and tour form are reached from the dashboard. The reusable navbar at the top of every authenticated screen exposes logout.

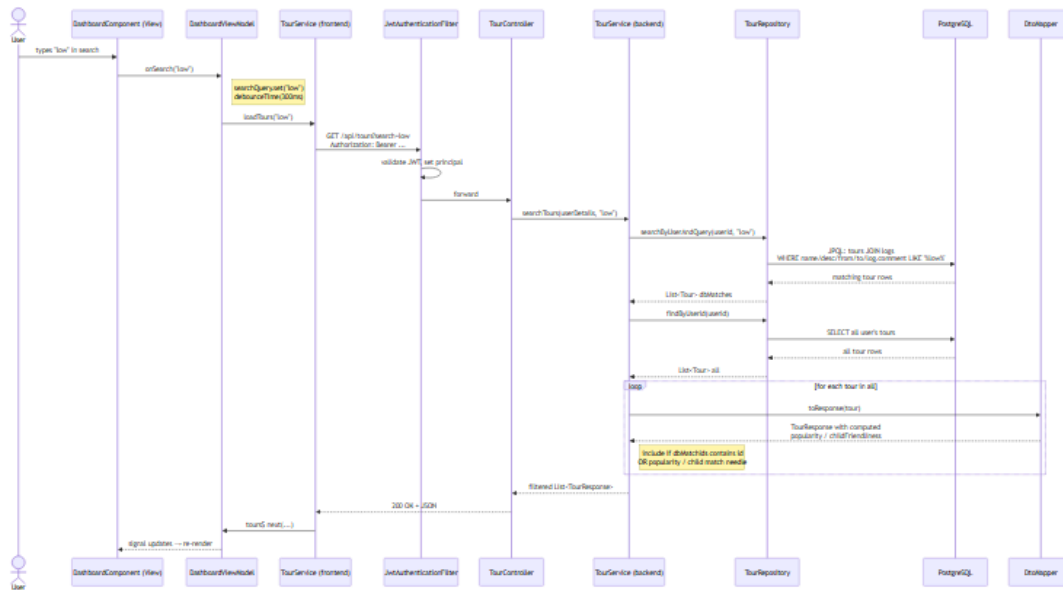


Wireframes are reproduced from the intermediate protocol in `TourPlanner_Protocol.pdf`. The final UI matches them, with the addition of:

- The **WEATHER AT DESTINATION** card (a blue gradient panel under the stats grid in Tour Detail) showing temperature, condition, humidity, wind, and the OpenWeatherMap icon — the *unique feature*.
- Wired-up **IMPORT** and **EXPORT** buttons on the dashboard with an inline error banner for parse / server failures.
- A search placeholder that reflects the actual scope: name, description, route, log comment, popularity.

5. Sequence Diagram — Full-Text Search

The user types in the dashboard search box. The ViewModel debounces 300 ms before firing a backend request. The backend uses a JPQL query for the stored fields and performs an in-memory pass for the computed popularity / child-friendliness.



Implementation references:

- `frontend/src/app/features/tours/dashboard/dashboard.viewmodel.ts:31` — `debounce`.
- `frontend/src/app/core/services/tour.service.ts:18` — `loadTours(search?)`.
- `src/main/java/org/example/tourplanner/repository/TourRepository.java:14` — JPQL.
- `src/main/java/org/example/tourplanner/service/TourService.java:55` — `searchTours`, including the in-memory pass for computed values.
- `src/main/java/org/example/tourplanner/dto/DtoMapper.java:55` — popularity formula; `:62` — child-friendliness formula.

6. Design Patterns Implemented

Pattern	Where	Why it's there
Repository	TourRepository, TourLogRepository, UserRepository extend JpaRepository<Entity, Long>	Hides JDBC; gives derived queries (findByUserId) and JPQL (searchByUserAndQuery) for free.

DTO + Mapper	<code>dto/*Request.java, dto/*Response.java, dto/DtoMapper.java</code>	Keeps JPA entities out of the HTTP surface; centralizes computed-attribute logic.
Builder (Lombok)	<code>@Builder</code> ON <code>Tour, TourLog, User, TourResponse, TourLogResponse, TourExportDto, WeatherResponse</code>	Readable test fixtures and chained construction; e.g. <code>Tour.builder().name(...).build()</code> .
Strategy	<code>OpenRouteService.PROFILE_MAP</code> maps <code>TransportType</code> → ORS profile string (HIKING → "foot-hiking", etc.)	Adding a transport type is a one-line map entry, no switch to maintain.
Dependency Injection	<code>@RequiredArgsConstructor</code> + Spring DI everywhere (<code>TourService</code> , controllers, etc.)	Constructors take collaborators as <code>final</code> fields; trivial to mock in <code>@ExtendWith(Mockito)</code> .
Filter chain	<code>JwtAuthenticationFilter</code> extends <code>OncePerRequestFilter</code> plugged into <code>SecurityFilterChain</code> in <code>SecurityConfig</code>	Token validation runs <i>before</i> every controller; controllers stay clean of auth concerns.
MVVM (frontend)	Each feature: <code>*.component.ts</code> (View) + <code>*.viewModel.ts</code> (ViewModel with <code>signal()</code> state)	Components only render; state lives in the ViewModel; testability and zoneless CD friendliness.
Singleton (Spring)	All <code>@Service</code> / <code>@Component</code> beans	Single shared instance per <code>ApplicationContext</code> ; standard Spring lifecycle.

7. Unique Feature — Weather at Destination

The application integrates the OpenWeatherMap API to display live weather conditions at each tour's destination. The feature is implemented end-to-end:

- **Backend:** `WeatherService.java` hits `https://api.openweathermap.org/data/2.5/weather` with the geocoded location, parses the response into a `WeatherResponse` DTO, and exposes it via `GET /api/tours/{tourId}/weather`. If `WEATHER_API_KEY` is not configured the service falls back to plausible stub data (so the demo works offline).
- **Frontend:** `TourDetailViewModel.loadWeather()` fetches the weather when the detail page opens; the result populates a `weather` signal. The "WEATHER AT DESTINATION" card in `tour-detail.component.html` renders temperature, description, the OpenWeatherMap icon, humidity, and wind speed.
- **Resilience:** failures are non-blocking — the rest of the page is fully functional without a weather card, so the load is wrapped in `subscribe({ error: ... })` that silently clears the signal.

Why this is the unique feature: it goes beyond the spec's required tour management features, integrates a third external service in addition to `OpenRouteService`, and demonstrates a graceful-degradation pattern that's useful in real applications.

8. Library Decisions & Lessons Learned

- **JWT over server sessions:** stateless tokens fit the SPA + REST architecture, let the backend stay horizontally scalable, and avoid CORS/cookie complications during local development. The downside (revocation requires a denylist) is acceptable for a personal tour app.

- **Lombok:** `@Data` , `@Builder` , `@RequiredArgsConstructor` cut significant boilerplate from entities, DTOs, and services. The cost is the extra annotation processor in the build, which we mitigate by excluding Lombok from the runnable JAR in the `spring-boot-maven-plugin` configuration.
- **Angular signals + zoneless change detection:** cleaner mental model than RxJS in every component. Templates bind to `signal()` s with one-way `[ngModel]="vm.x()" / (ngModelChange)="vm.x.set($event)"` — keeps the ViewModel as the single source of truth.
- **Leaflet over Google Maps:** open-source, OpenStreetMap-friendly, no API key needed, and it integrates with the GeoJSON returned by OpenRouteService directly via `L.geoJSON()` .
- **OpenRouteService quirk:** directions need lon/lat, not place names, so each tour create/update first geocodes both ends and then fetches directions — two calls per save. Our `OpenRouteService` class handles this in one method to keep callers simple.
- **In-memory filter for computed values:** full-text search over popularity / child-friendliness can't be expressed in SQL because they're derived from logs. Splitting the work — JPQL for stored fields, in-memory pass for derived ones — keeps the search responsive while staying correct.
- **Import idempotency:** silently skipping duplicate names on import was deliberate. Re-importing the same file is a common pattern; clobbering the user's edits would be surprising, while a 409 would block bulk imports for one bad row.

9. Unit Tests — Why these and what they protect

36 backend tests across 9 test files. Each test pins down one specific contract; together they cover every layer that contains business logic.

File	#	What it verifies
<code>AuthControllerTest</code>	2	<code>POST /register</code> and <code>POST /login</code> return 200 with the auth response shape
<code>TourControllerTest</code>	3	<code>GET /tours</code> , <code>GET /tours/{id}</code> , <code>POST /tours</code> produce the expected JSON
<code>AuthServiceTest</code>	4	Register hashes password, login validates password, duplicates are rejected, missing user 401
<code>TourServiceTest</code>	8	List, get, create, update, delete; ownership check; ORS-result mapping into entity
<code>TourServiceSearchTest</code>	4	NEW Search hits name, description, computed popularity; blank query returns all (no JPQL call)
<code>TourServiceImportExportTest</code>	3	NEW Export round-trips logs; import creates tour+logs; import skips duplicate names
<code>TourLogServiceTest</code>	5	Log CRUD; log scoped to its tour; ownership check
<code>DtoMapperTest</code>	4	Tour → response mapping; popularity threshold (0/1/5/10); child-friendliness from logs
<code>JwtTokenProviderTest</code>	4	Token signs, verifies, extracts subject, rejects tampered/expired tokens
Total	36	

Why these tests matter:

- **Service-layer tests** (Tour/TourLog/Auth) are the highest-value tests: they verify the business rules that the rest of the system trusts. A regression here would let tours cross user boundaries or accept invalid log statistics.
- **DtoMapperTest** is critical because the popularity / child-friendliness *formulas* are the one place where the spec requires "computed attributes" — if the formula silently breaks, search and the UI both lie.
- **JwtTokenProviderTest** verifies the security boundary: any test that pretends the user is authenticated relies on this contract holding.
- **TourServiceSearchTest** locks in the dual-source search semantic (JPQL + in-memory pass for computed values) — it's the load-bearing piece of the rubric line "search also considers the computed values".
- **TourServiceImportExportTest** prevents two specific failure modes: silent log loss on round-trip, and silent overwrite on duplicate import.

Test quality choices: prefer pure unit tests with `@ExtendWith(MockitoExtension.class)` over integration tests. Controller tests use `@WebMvcTest` with mocked services so they boot in milliseconds. No tests share fixtures across classes; every test sets up its own state in `@BeforeEach`.

10. Tracked Time

Estimated from the project's git history (33 commits spanning Feb 16 – May 18, 2026). Numbers are best-effort and should be refined with real records.

Phase	Dates	Hours
Project setup; backend domain model + repositories	Feb 16 – Feb 21	10
Auth + JWT + DTOs	Feb 23 – Feb 26	8
Services: TourService, OpenRouteService, WeatherService	Feb 25 – Feb 28	8
REST controllers + GlobalExceptionHandler	Feb 28 – Mar 1	6
Backend unit tests (initial set)	Mar 1	5
Frontend init: project, models, core services	Mar 2 – Mar 3	6
Auth UI + Navbar + reusable TourMap component	Mar 4 – Mar 5	6
Dashboard, Tour Detail, Tour Form (create/edit)	Mar 6 – Mar 9	12
Intermediate hand-in feedback (search, map, nav fixes)	Apr 3 – Apr 9	6
Final features: full-text search expansion, import/export, weather UI, navbar consistency	Apr – May	10
Logging, configuration externalization, bug fixes	May 17 – May 18	4
Final protocol: UML diagrams, design pattern map, write-up	May	6
Total estimate		~87

11. Git Repository

https://github.com/Many-Attempts/tour_planner.git

The git history is part of the documentation: every commit message describes the incremental step (`add Tour` and `TourLog domain models` , `add OpenRouteService` and `weather API integration` , `add backend unit tests` , etc.). The intermediate hand-in is reachable at the commit immediately before the final-submission work begins (`10ada24 fixed nav`).